CampusSlot

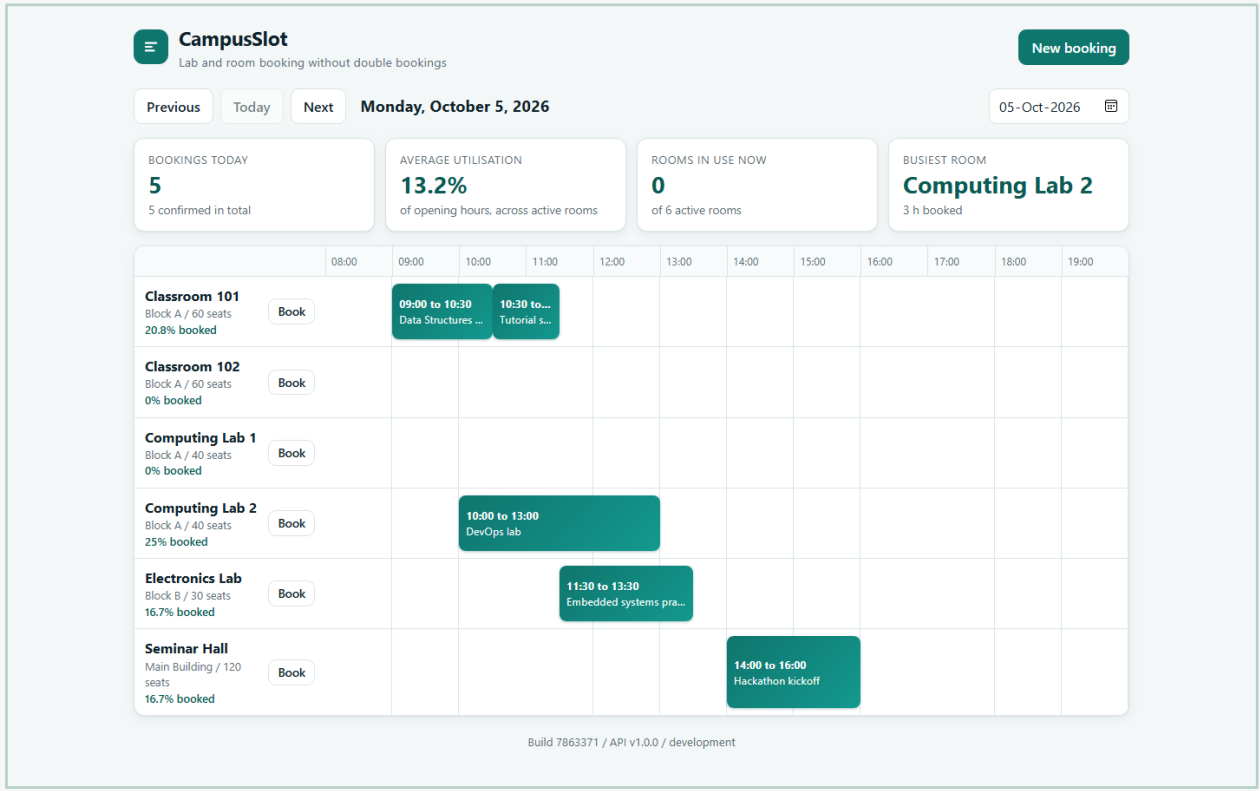
Book a campus room or lab without ever double booking it

The problem

Rooms are often booked by message or spreadsheet, and two people end up with the same room at the same time.

The idea

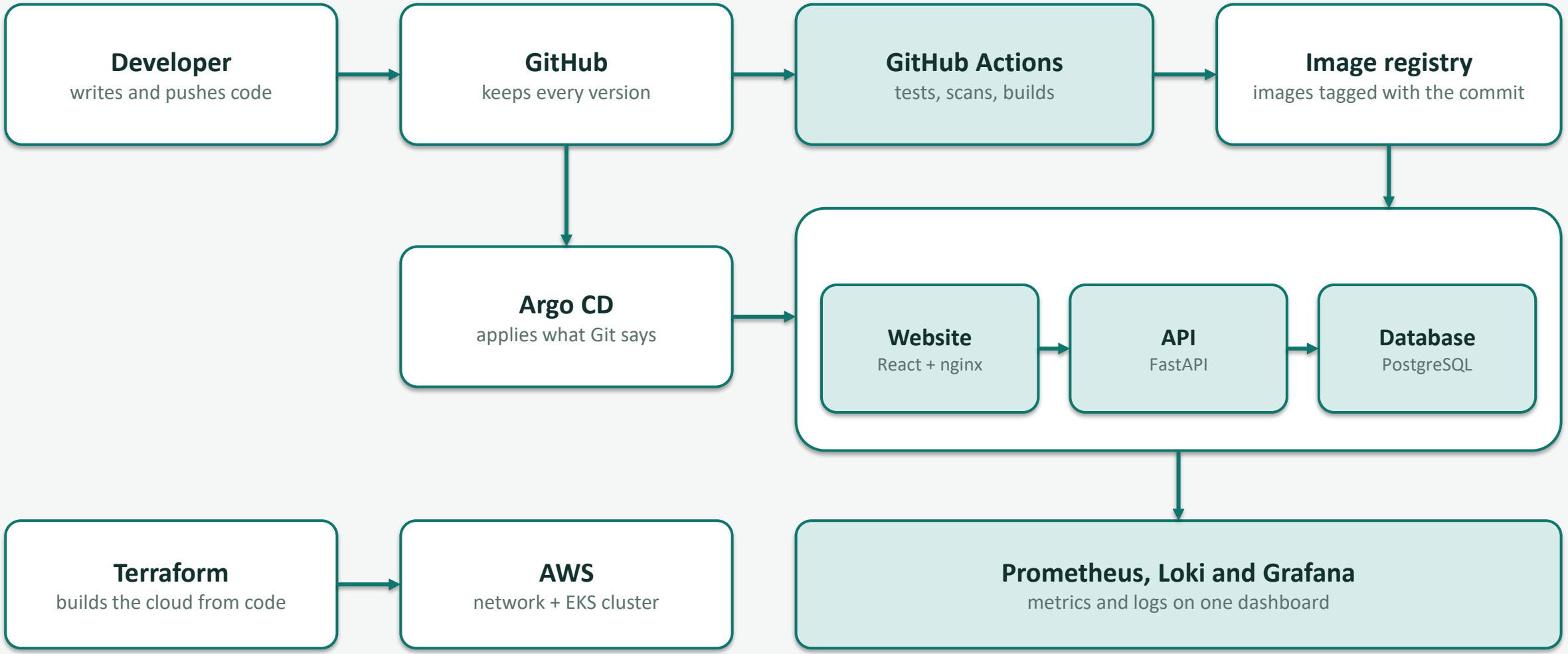
A booking system that refuses an overlapping slot, and a delivery chain around it that is automatic, secure and observable.



Ujjwal Jain | 24bcs10173 | Section B
github.com/Ujjwaljain16/campusslot

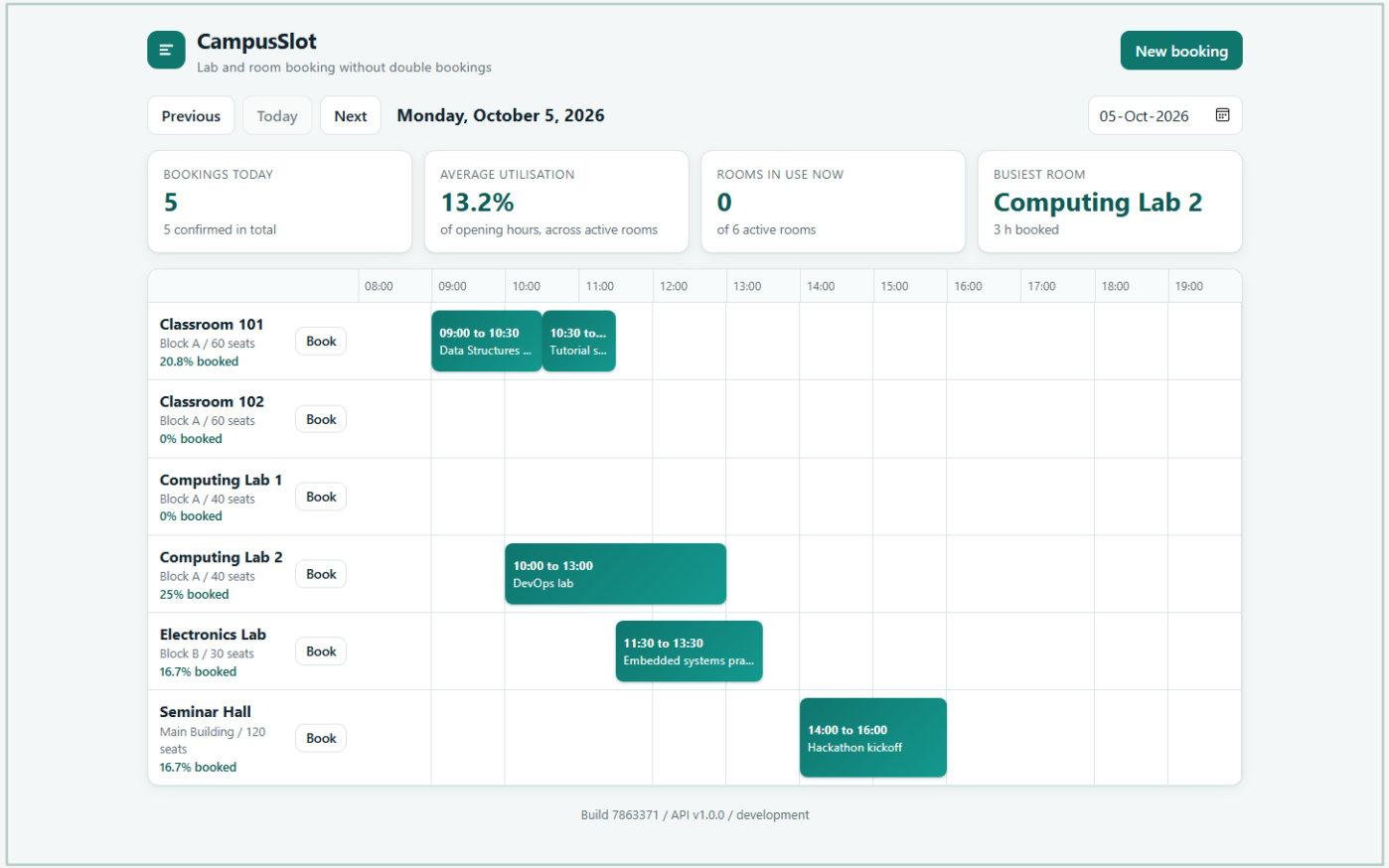
How the pieces fit together

From a developer's keyboard to a running, watched application



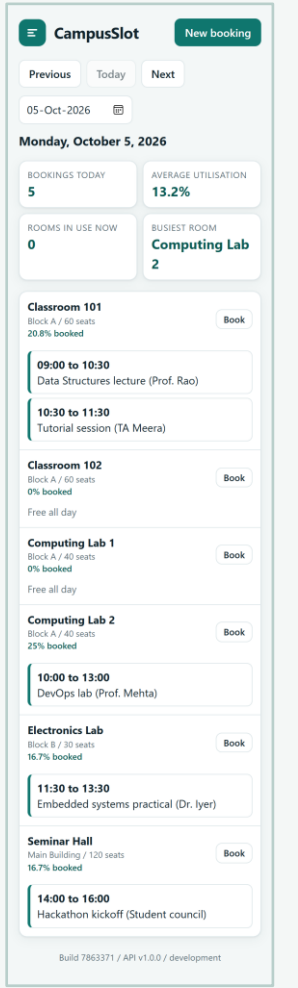
What the application does

Pick a room and a time. If the slot is taken, the system says no and says why.



The day view for six rooms, with live numbers on top

Real data: bookings are stored in PostgreSQL and survive restarts.



On a phone

Three layers protect the rule

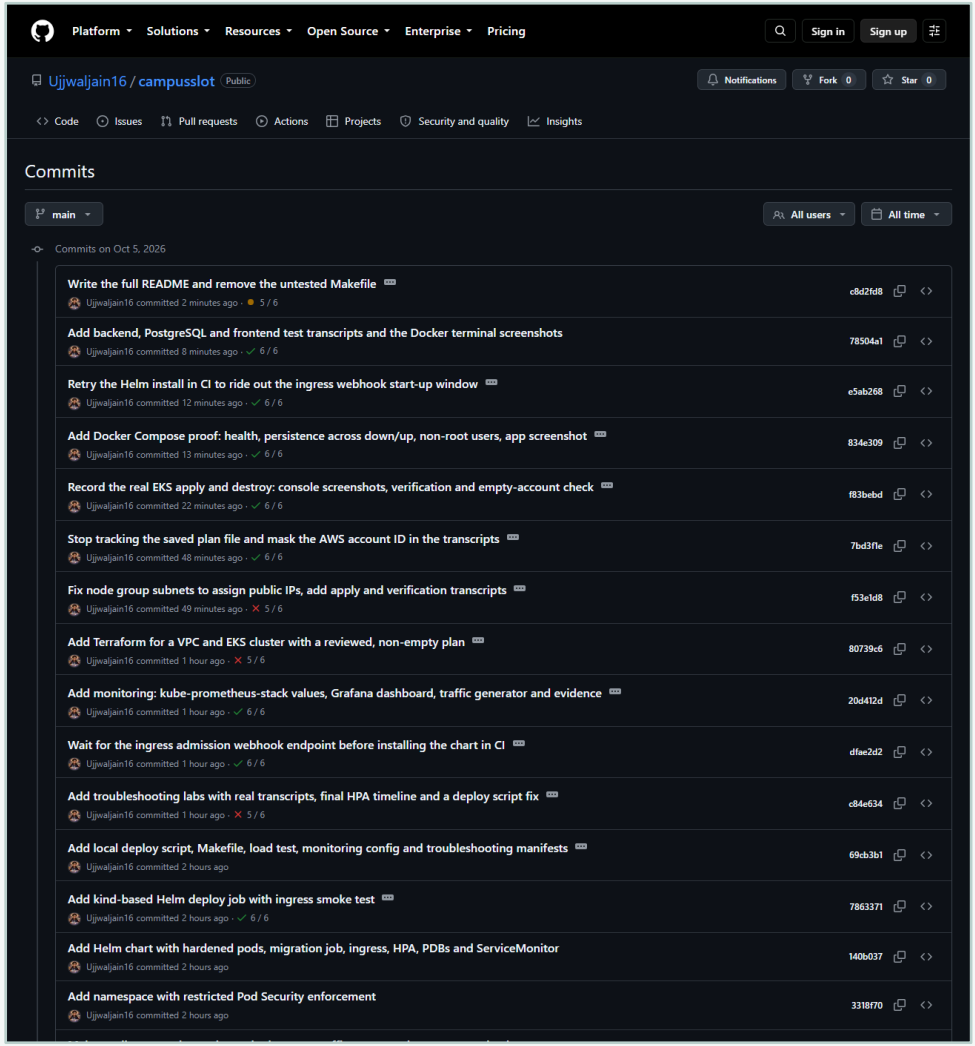
1. Check the request makes sense
2. Look for an overlapping booking
3. The database refuses overlaps itself

A slot that ends when another begins is fine.

How the project is organised

More than 80 meaningful commits, each one a small step that can be explained

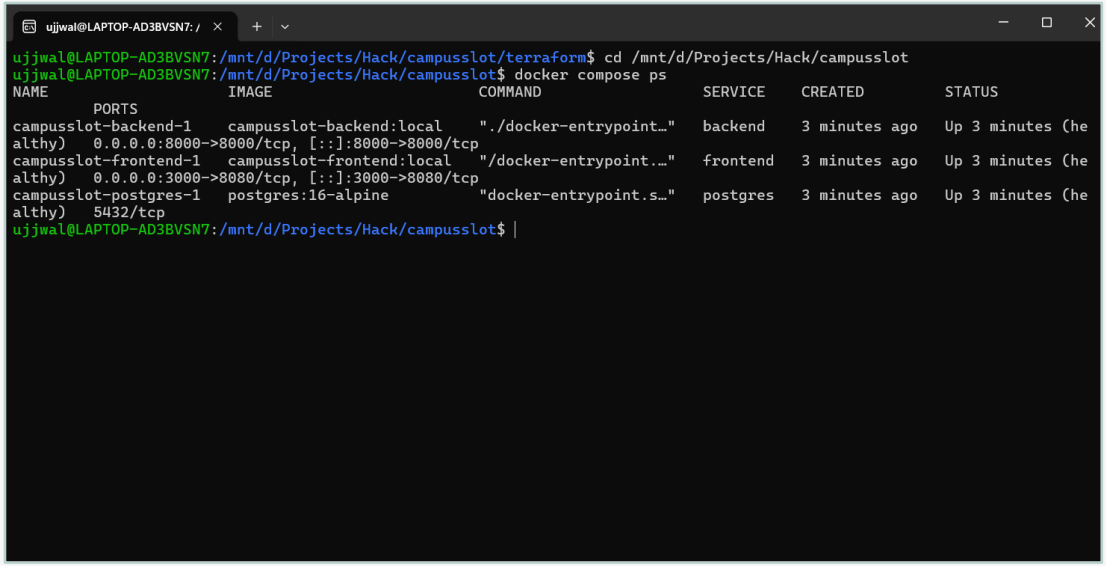
```
backend/          API, migrations, tests
frontend/         website and nginx
helm/             the Kubernetes package
terraform/        AWS network and cluster
gitops/           what Argo CD applies
security/         the security gates
monitoring/       dashboards and logs
troubleshooting/   four failures on purpose
docs/evidence/ real screenshots and logs
.github/workflows/  the pipeline
```



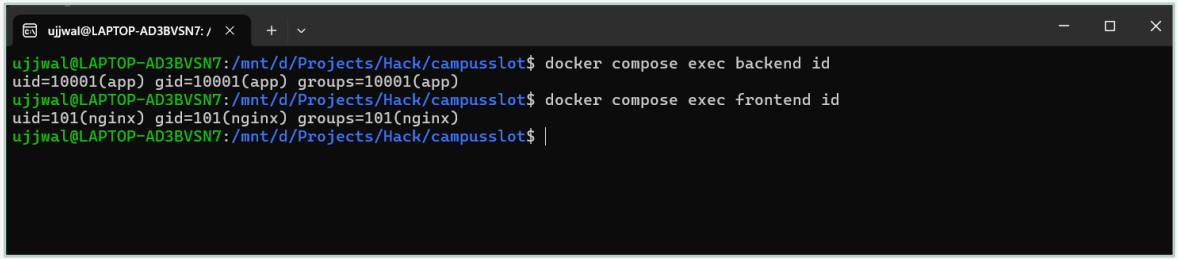
Public commit history on GitHub

One command starts everything

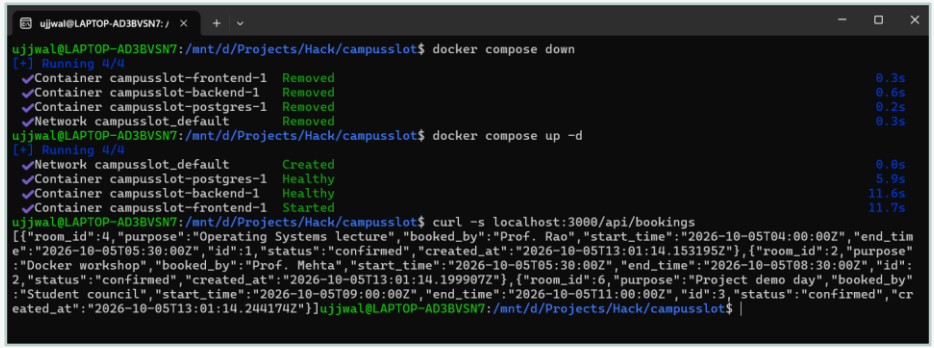
docker compose up: website, API and database, with the data kept safe



Three services, all healthy



Both containers run as a normal user, not as root

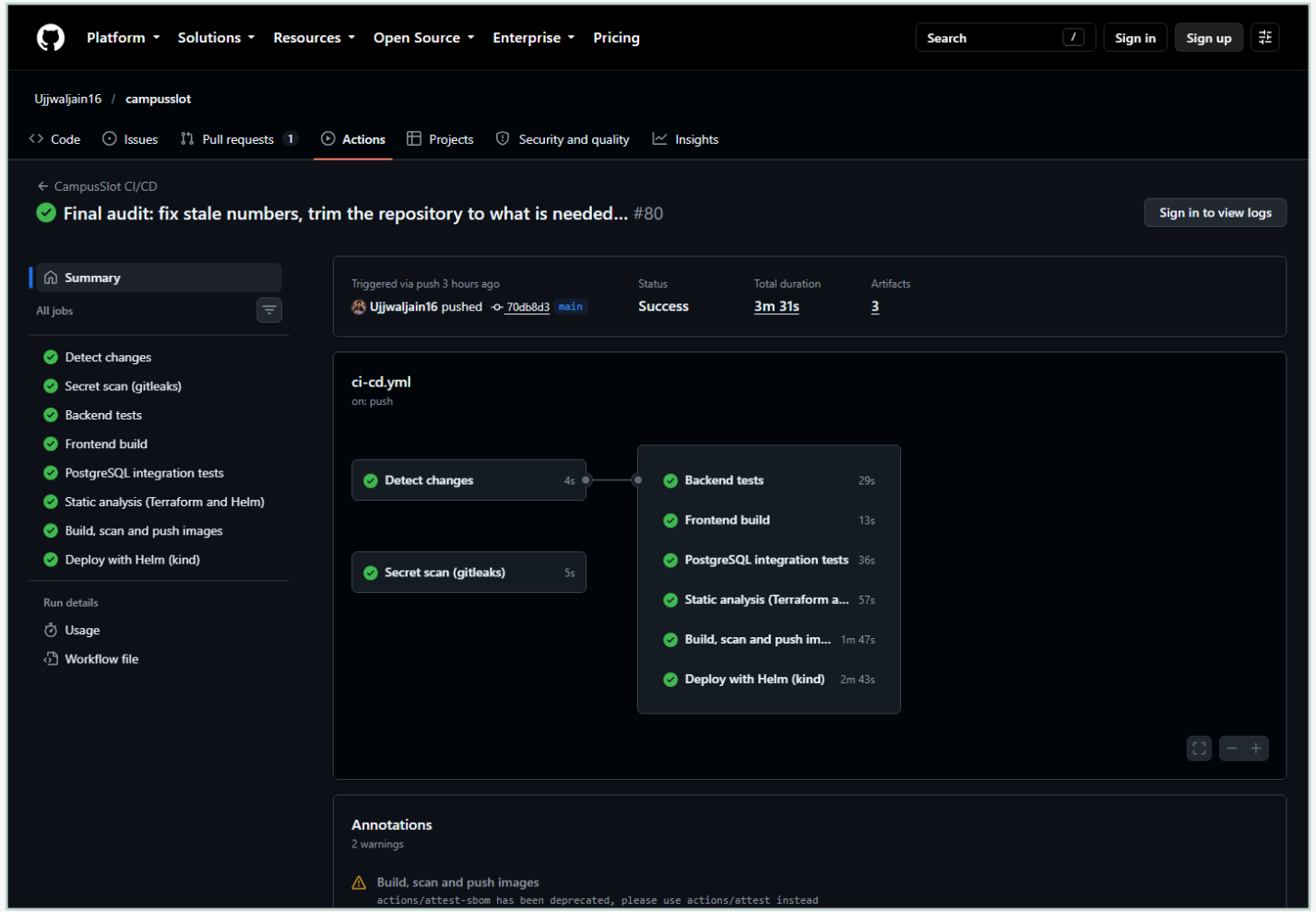


Bookings survive down and up

- Two small images, built in stages
- Health checks on every service
- The database keeps its data on a volume

Every push is checked automatically

Nothing reaches the cluster unless every earlier check has passed

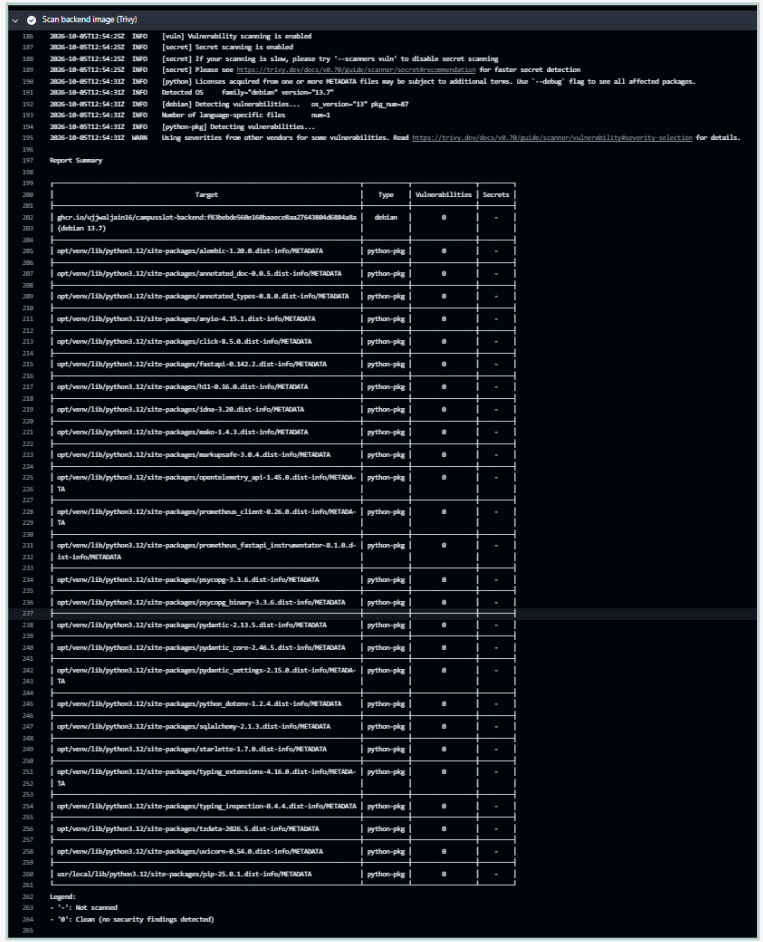


Run in the screenshot	Time
Backend tests	19 s
Frontend build	9 s
Database tests	34 s
Secret scan	7 s
Build, scan, push	98 s
Deploy and smoke test	98 s

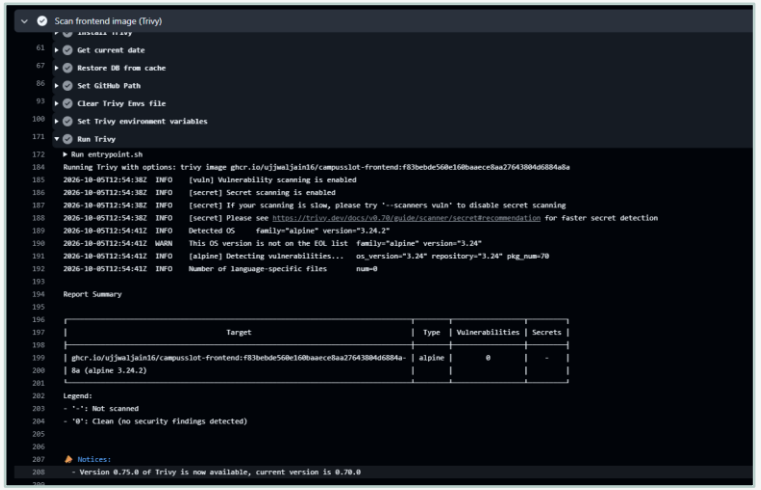
- Runs on every push to main
- Images are tagged with the commit, never latest
- Today: jobs side by side, about 3 minutes

Security checks before anything ships

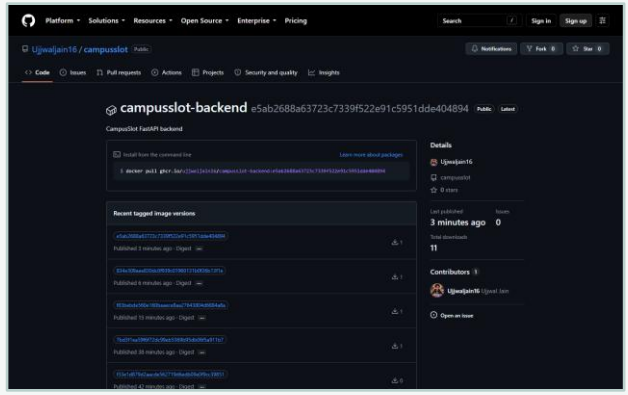
Both images scanned: zero serious findings



Backend image: 0 vulnerabilities



Frontend image: 0 vulnerabilities



Published in GHCR, tagged with commits

Six kinds of check

Code: Bandit

Libraries: pip-audit, npm audit

Secrets in history: Gitleaks

Images: Trivy, blocks the build

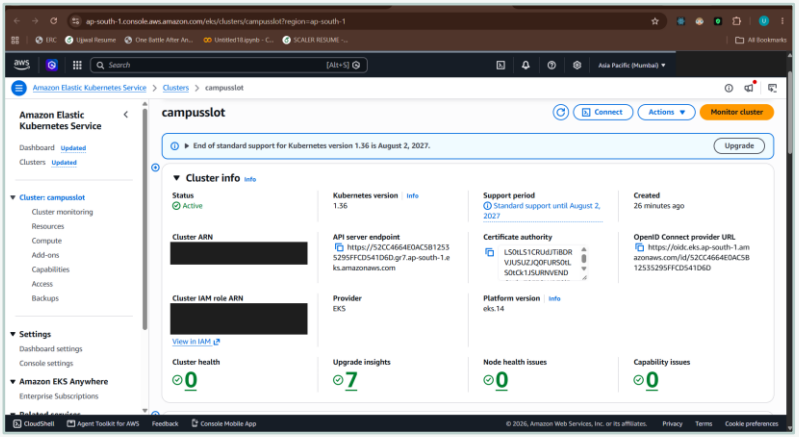
Terraform and Helm: Trivy

Origin: signed provenance

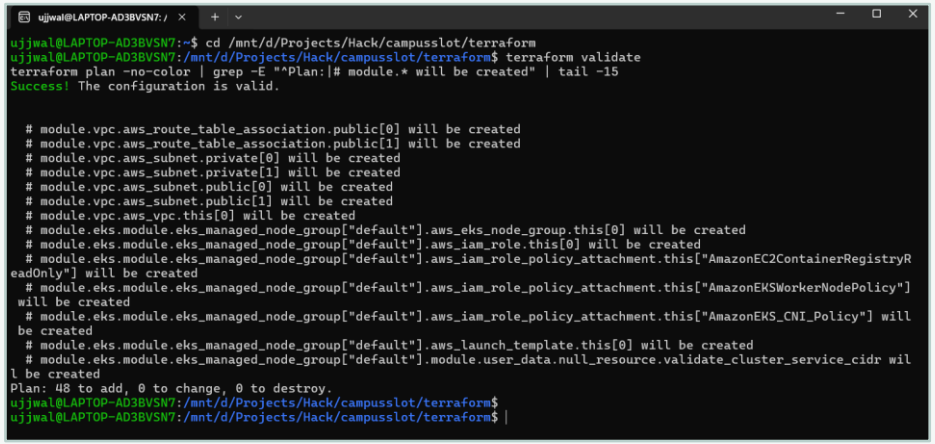
A clean scan is not proof of safety. It means nothing fixable was known that day.

Cloud infrastructure built from code

A real network and Kubernetes cluster on AWS, built and then removed



EKS cluster: Active



Plan: 48 resources to add

Built, checked, destroyed

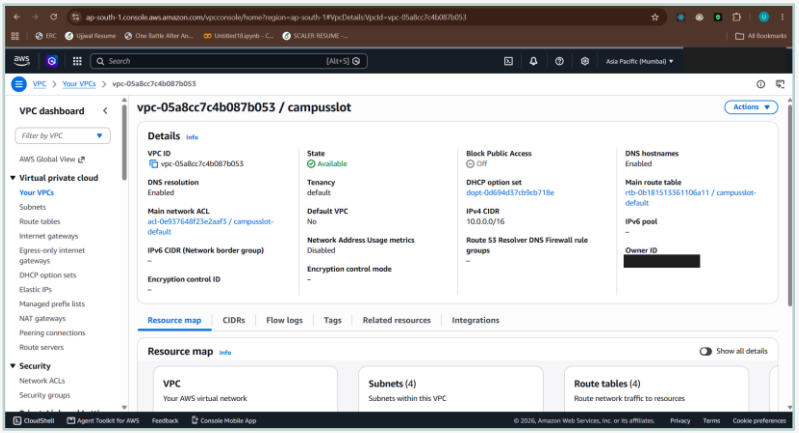
48 resources from code

Ran for about 45 minutes

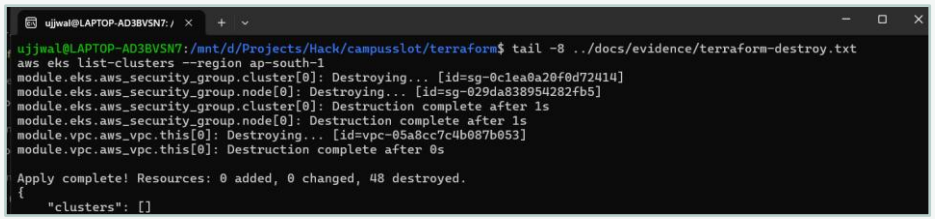
About 6 US cents

Account checked: nothing left

First apply failed. I fixed the cause and wrote it down.



The network (VPC)



Destroyed: 48 resources removed

Running on Kubernetes with Helm

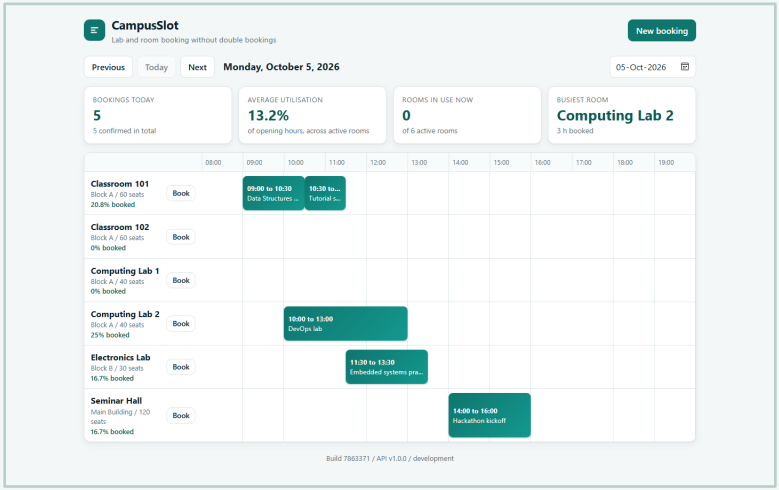
Two copies of the website and API, one entrance, and automatic scaling

```
Windows PowerShell
PS D:\Projects\Hack\campusslot> kubectl get pods -n campusslot
NAME                                 READY   STATUS    RESTARTS   AGE
campusslot-backend-575d569c49-lv2wh   1/1     Running   0          137m
campusslot-backend-575d569c49-s8j2k   1/1     Running   0          137m
campusslot-frontend-76c977dcc-69gdm   1/1     Running   0          137m
campusslot-frontend-76c977dcc-88hlj   1/1     Running   0          137m
campusslot-postgres-0                 1/1     Running   0          137m
PS D:\Projects\Hack\campusslot> kubectl get svc -n campusslot
NAME                TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)    AGE
campusslot-backend    ClusterIP   10.97.110.226    <none>        8000/TCP   137m
campusslot-frontend   ClusterIP   10.102.202.238   <none>        80/TCP     137m
campusslot-postgres   ClusterIP   10.104.182.44    <none>        5432/TCP   137m
PS D:\Projects\Hack\campusslot> helm list -n campusslot
NAME            NAMESPACE       REVISION        UPDATED                                 STATUS          CHART                   APP VERSION
campusslot      campusslot      3               2026-10-05 17:10:33.5040801 +0530 IST   deployed        campusslot-1.0.0        1.0.0
PS D:\Projects\Hack\campusslot> kubectl get ingress,hpa -n campusslot
NAME                                CLASS   HOSTS   ADDRESS        PORTS   AGE
ingress.networking.k8s.io/campusslot   nginx   *       192.168.58.2   80      137m

NAME                                                        REFERENCE                      TARGETS       MINPODS   MAXPODS   REPLICAS   AGE
horizontalpodautoscaler.autoscaling/campusslot-backend   Deployment/campusslot-backend   cpu: 3%/50%   2         5         2          137m
PS D:\Projects\Hack\campusslot>
```

Pods running, services, Helm release, ingress and autoscaler

Autoscaling test, first sizing	What happened
Load starts	2 API copies, CPU 3 percent
Under load	CPU 428 percent, grows to 5 copies
Load stops	Back to 2 copies about four minutes later

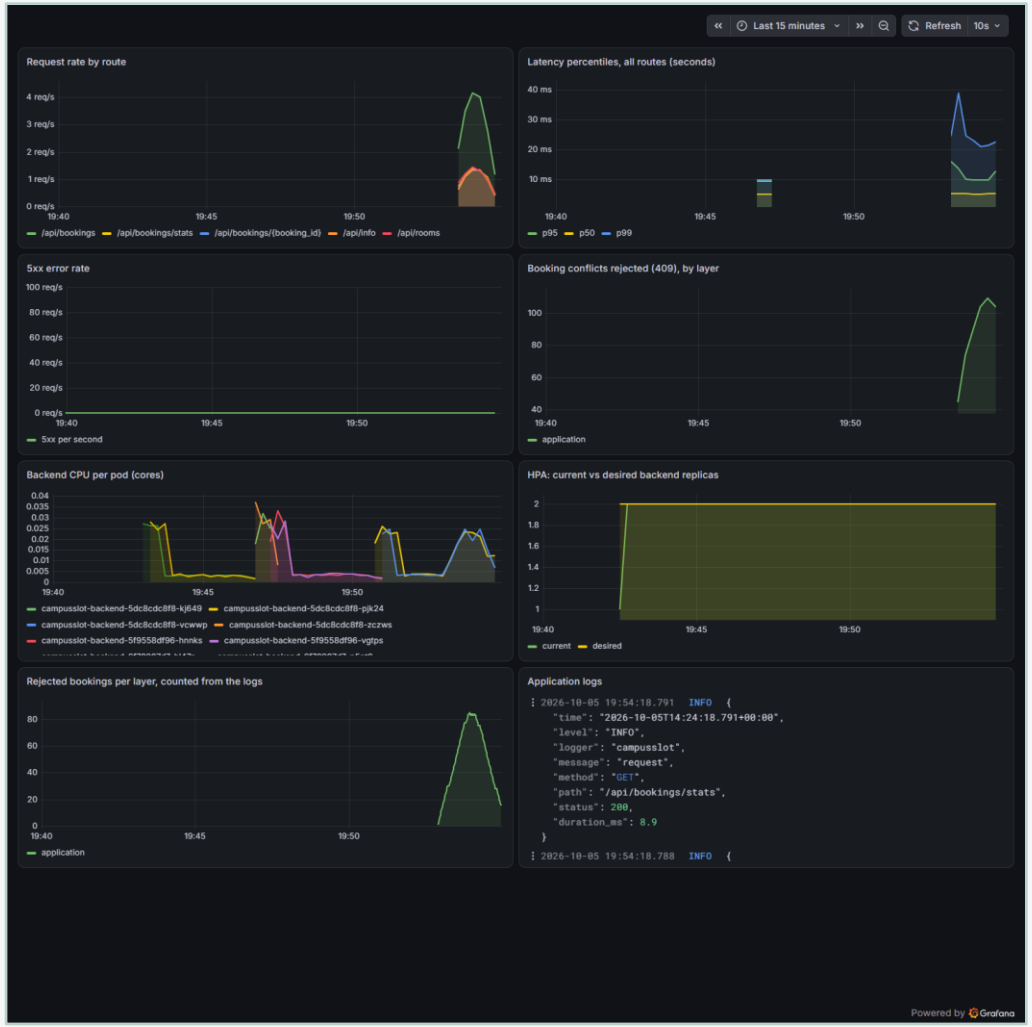


The application, reached through the ingress

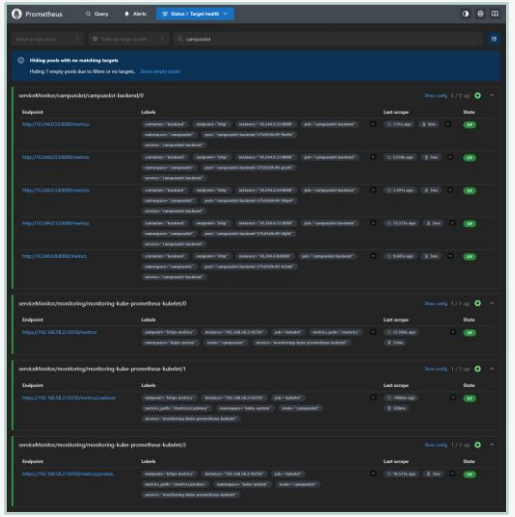
- Settings in a ConfigMap
- Password in a Secret, never in Git
- One entrance: / and /api

Watching the system run

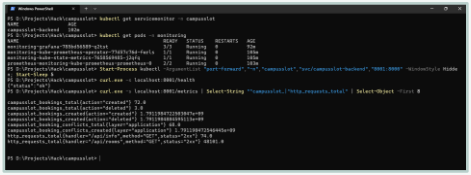
Metrics and logs together on one dashboard



Grafana: requests, speed, errors, conflicts, CPU, scaling and logs



Prometheus: every API copy is up

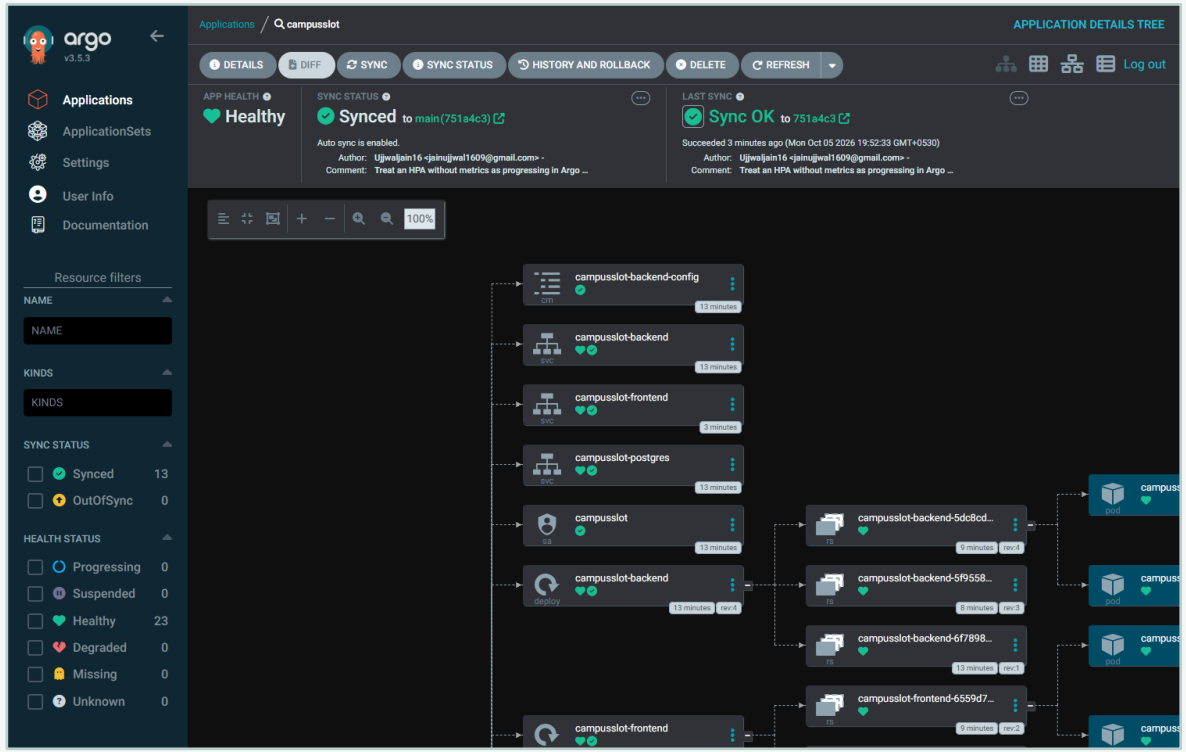


The metrics endpoint

- Metrics: Prometheus
- Logs: Loki, collected by Alloy
- Check: 105 conflicts created, 105 counted in the logs

Self-healing deployment, and fixing broken things

Git is the source of truth, and I practised four real failures



Argo CD: Synced and Healthy

- A commit changes the cluster
- Manual changes are undone in seconds

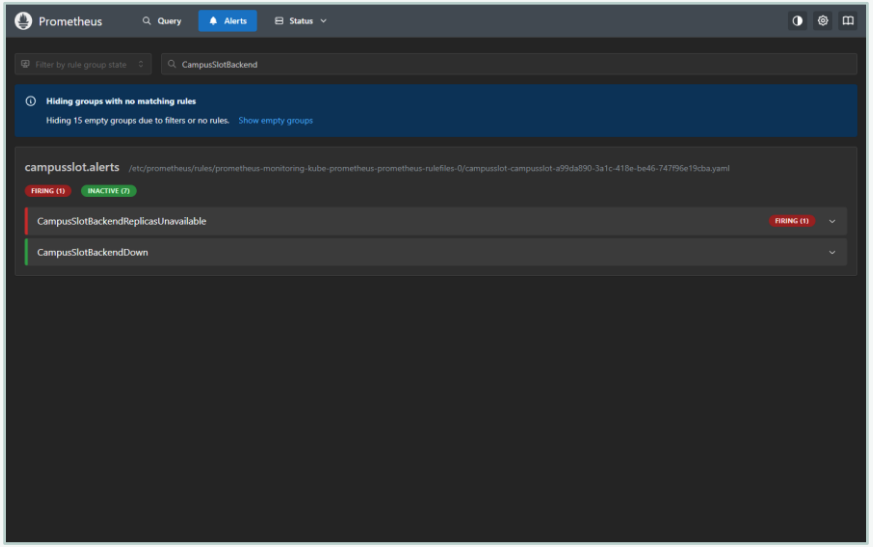
What I broke	What I found
Wrong image name	Image cannot be pulled
Typo in a selector	Service has no endpoints
Wrong database address	Pod keeps restarting
Same address, no migration	Pod runs but is not ready

- Each failure: identify, investigate, root cause, fix, verify.
- Real transcripts for all four are in the repository.

Measure, change one thing, measure again

Every improvement has a number from before and a number from after

What I measured	Before	After
Lost requests in a rollout	8 of 8 rollouts	0 of 6
Database errors seen by alerts	0 of 18	16 of 16
Alert on a broken deploy	none	fires in 185 s
Pipeline, median	238 s	148 s
Restore after a mass delete	no backup	identical, 4 s
Broken release recovery	by a person	automatic, 72 s



Prometheus: the alert firing during the drill

- The method: baseline, one change, the same measurement again, then write down the trade-off.
- The samples are small and from one laptop. The documents in the repository say so.

Decisions, trade-offs and what I left out

Knowing what not to build is part of the job

8 s longer per pod in every rollout
the price of losing no requests

About a minute longer per pipeline run
the price of signed, verified images (143 s vs 202 s and 214 s)

Backup stays in the same cluster
covers mistakes, not the loss of the cluster

One database pod
a 6 s outage remains, now visible to the alerts

- **Not done, on purpose**
- Git revert vs Argo CD rollback
- Automated promotion
- Network policies
- Alert delivery (no Alertmanager)
- Write-heavy load test
- Why two workers gave no gain
- Backups outside the cluster

14 decision records in docs/adr explain the choices and what I would do in production.

Does it hold up when it grows?

I loaded 200 thousand bookings, loaded one pod to its limit, and sized the pods from measurements

What I measured	Before	After
Pages read, overlap check, 200k rows	2469	30
Day view, all rooms, 200k rows	21 req/s	113 req/s
One backend pod, room list	153 req/s	342 req/s
Pods needed for 60 req/s	5	2
Memory reserved for those pods	640 MiB	192 MiB

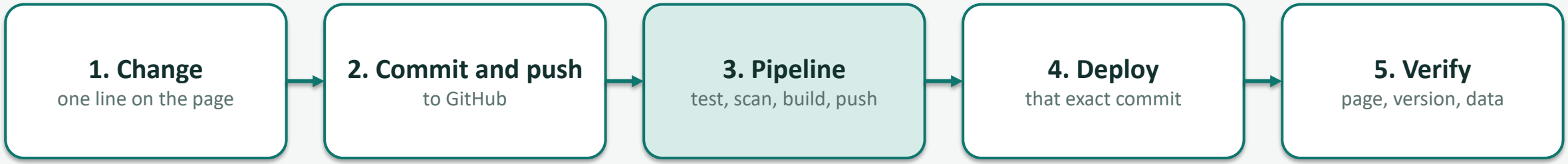
- **Tried and rejected**
- Two workers: no gain, twice the memory, wrong metrics
- ETag on the room list: no gain
- Both removed from the code

What the numbers taught me

- The CPU limit that I wrote without measuring was the first bottleneck.
- Right-sizing did not shrink everything: the CPU request went up, because the old one was too low.
- Two ideas that I expected to help did nothing, so I removed them from the code.

From a change to a running system

I rehearsed the whole loop, and here is what I learned



Result	Number
Automated tests	47 + 7 + 12
Serious vulnerabilities	0
Cloud resources built and removed	48
Pipeline, full green run	about 3 minutes

In my rehearsal the first run failed in my own test. I fixed the test, then the new version went live and I checked it from the outside.

What I learned

Test the system as it really runs.

Write the failures down, they teach the most.

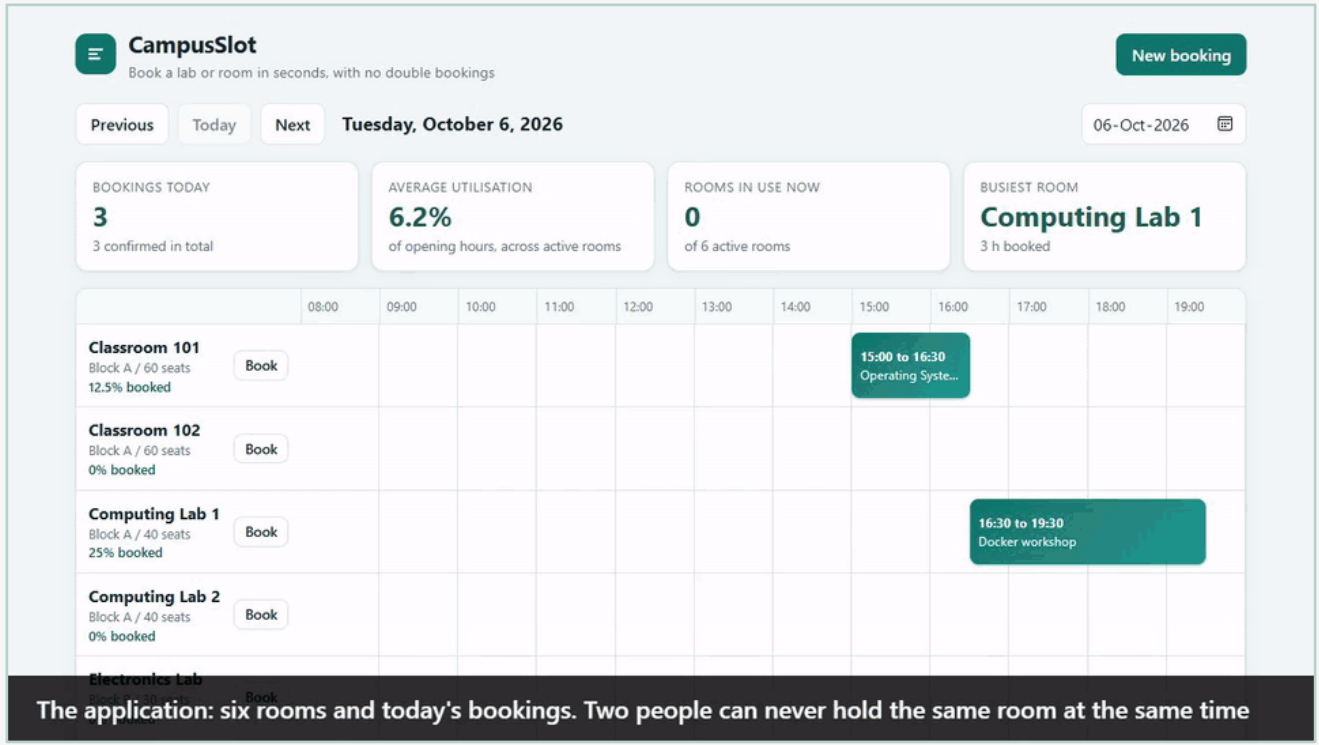
A clean scan is not proof of safety.

Cost control is part of the design.

Git as the source of truth makes changes visible and reversible.

See it run: the recorded demo

A broken release is rolled back by itself, and no user request fails



A 33 second highlight of the recording. The failure drill is sped up about ten times.

5.7 s
first failed probe

63 s
automatic rollback

0 of 1,600
user requests failed

The full recording: 4 min 22 s, with captions

1. The application, and a double booking that is refused
2. The pipeline and its green runs
3. A broken release, deployed live and rolled back by itself
4. Prometheus sees the unavailable replica

Scan to watch the full recording

github.com/Ujjwaljain16/campusslot, docs/demo/CampusSlot-demo.mp4

Thank you

Everything on these slides is in the repository, with the transcripts behind every number

47 + 7 + 12
automated tests

0
serious vulnerabilities in the images

48
cloud resources built and removed

0 of 1,600
requests failed in the rollback drill

Where to look

Repository: github.com/Ujjwaljain16/campusslot

README: 20 sections, then the engineering results

Recorded demo: docs/demo/CampusSlot-demo.mp4

Evidence: docs/evidence, real screenshots and transcripts

Decisions: docs/adr, 14 decision records

Questions?

Ujjwal Jain | 24bcs10173 | Section B